DAY 10

Duplicate:

Duplicate data refers to repeated records within a dataset that represent the same entity or transaction. These duplicates can arise due to system errors, multiple data entries, or improper data merging. They can distort analysis by inflating counts, leading to inaccurate insights and poor decision-making if not handled properly.

Null:

Null values represent missing, undefined, or unknown data in a dataset. They occur when information is not collected, is unavailable, or fails to be recorded. Nulls can impact data quality and analysis because many algorithms and calculations cannot process missing values directly.

Delete Null Value:

Deleting null values involves removing records or fields that contain missing data. This approach is useful when the proportion of null values is small and unlikely to impact the dataset significantly. However, excessive deletion can lead to loss of important information and reduced dataset size.

Replace Null Value:

Replacing null values involves filling in missing data with substitute values such as mean, median, mode, or a default constant. This method helps maintain dataset size and enables smoother analysis, but it requires careful selection of replacement values to avoid introducing bias.

Transform:

Data transformation is the process of converting data into a suitable format or structure for analysis. It may include cleaning, aggregating, normalizing, or encoding data. Transformation improves data consistency, usability, and compatibility with analytical tools.

Simple Pipeline:

A simple pipeline is a sequence of data processing steps applied in a structured and repeatable manner. It typically includes stages such as data collection, cleaning, transformation, and analysis. Pipelines help automate workflows and ensure consistency in data handling.

Identify: How duplicates affect revenue calculation:

Duplicates can significantly distort revenue calculations by counting the same transaction multiple times. This leads to inflated revenue figures, incorrect financial reporting, and misguided business decisions. Identifying and removing duplicates is essential to ensure accurate revenue analysis.

Explain: Why null handling is important:

Handling null values is crucial because missing data can lead to errors, biased results, or incorrect conclusions in analysis. Proper null handling ensures data completeness, improves model performance, and enhances the reliability of insights derived from the dataset.

DAY 11

1. **select()** is used to choose specific columns or expressions from a DataFrame and returns a new DataFrame with only those columns.
   Example: `df.select("name", "age")` → Returns only *name* and *age*

2. **withColumn()** is used to add a new column or modify an existing one without removing other columns.
   Example: `df.withColumn("age_plus_1", df.age + 1)` → Adds a new column while keeping all existing columns

3. **alias()** is used to rename a column temporarily, usually in queries or transformations. It doesn't change the actual column name in the DataFrame schema.
   Example: df.select(df.name.alias("student_name")) → Displays column as *student_name* in output

4. **distinct()** removes duplicate rows considering all columns.
   Example: df.distinct() → Removes completely identical rows
   `distinct()` does NOT look at individual columns separately.

   For this data:
   id, name, age
   1, Alice, 20
   1, Alice, 20
   1, Alice, 21

   - Using distinct() - > df.distinct()

   distinct() checks all columns together.
   Result
   id, name, age
   1, Alice, 20
   1, Alice, 21

5. **dropDuplicates()** allows removing duplicates based on specific columns.
   Example: `df.dropDuplicates(["name"])` → Keeps only one row per unique *name*

   Example
   Data
   id, name, age

1, Alice, 20
1, Alice, 20
1, Alice, 21

- Using dropDuplicates() -> df.dropDuplicates()

Result:
id, name, age
1, Alice, 20
1, Alice, 21

- `df.dropDuplicates(["id", "name"])`
  Now only `id` and `name` are checked.
  Result could be:
  id, name, age
  1, Alice, 20
  One row is retained for each unique `(id, name)` pair.

6. Both **orderBy()** and **sort()** are used to sort a DataFrame. They are functionally the same in Spark.
   Example: `df.orderBy("age")`
   `df.sort("age")` → Both sort data by *age*

7. **limit()** is used to restrict the number of rows returned from a DataFrame.
   Example: `df.limit(5)` → Returns only the first 5 rows

8. **count()** returns the total number of rows in a DataFrame.
   Example: `df.count()` → Outputs total row count (e.g., 1000)

DAY 12

Date functions:

to_date() function in PySpark is used to convert a string column into a date format (`DateType`). It helps Spark perform date operations such as filtering, sorting, and extracting year or month values. It is commonly used when dates are stored as strings in datasets.

```
from pyspark.sql.functions import to_date
df = df.withColumn("date", to_date("date_string", "yyyy-MM-dd"))
```

month() function extracts the month value from a date column and returns an integer between 1 and 12. It is useful for monthly analysis, reports, and grouping data based on months.

```
from pyspark.sql.functions import month
df = df.withColumn("month", month("date"))
```

year() function in PySpark is used to extract the year value from a date or timestamp column. It returns the year as an integer and is useful for yearly reports, trend analysis, and grouping data based on years.

```
from pyspark.sql.functions import year
df = df.withColumn("year", year("date"))
```

dayofmonth() function extracts the day number from a date column. It returns values from 1 to 31 and is useful for date-wise tracking, attendance systems, and billing applications.

```
from pyspark.sql.functions import dayofmonth
df = df.withColumn("day", dayofmonth("date"))
```

String functions:

lower() function converts all characters in a string column into lowercase letters. It is mainly used for data cleaning and case-insensitive comparisons.

```
from pyspark.sql.functions import lower
df = df.withColumn("lower_name", lower("name"))
```

upper() function converts all characters in a string column into uppercase letters. It helps standardize text data and is commonly used in reporting and formatting.

```
from pyspark.sql.functions import upper
df = df.withColumn("upper_name", upper("name"))
```

Trim() function removes extra spaces from the beginning and end of string values. It is useful for cleaning data before filtering, joining, or analysis.

```
from pyspark.sql.functions import trim
df = df.withColumn("clean_name", trim("name"))
```

Concat() (using lit in between): `concat()` function combines multiple columns into one string column, while `lit()` adds constant values like spaces or symbols during concatenation. It is useful for creating formatted text such as full names or addresses.

```
from pyspark.sql.functions import concat, lit
df = df.withColumn("full_name", concat("first_name", lit(" "), "last_name"))
```

What is the difference between applying the `month()` function directly on a date string and applying it after converting the string using `to_date()` in PySpark?

When the `month()` function is applied directly to a string column, Spark tries to automatically interpret the string as a date. This works only if the string is in a recognized date format such as `yyyy-MM-dd`. However, if the format is inconsistent or unsupported, it may produce errors or null values.

When the string is first converted using `to_date()`, the column becomes a proper `DateType`. Applying `month()` after conversion is more reliable, readable, and safer for real-world datasets because Spark clearly understands the data as a date.